

Convoy Effect

Definition:

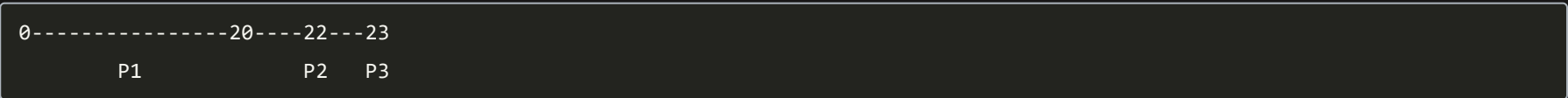
The **Convoy Effect** occurs when **one long CPU-bound process holds the CPU**, forcing many short processes to wait behind it, resulting in poor CPU utilization and increased waiting time.

Example

Processes

P1 : BT = 20
P2 : BT = 2
P3 : BT = 1

Using **FCFS**:



Even though **P2** and **P3** are very short, they must wait **20 units** for P1 to finish.

Why is it bad?

- High average waiting time
- Poor response time
- Interactive users experience delays
- I/O devices may remain idle while waiting for the long CPU-bound process to release the CPU

Which algorithm suffers from it?

- **FCFS** (most commonly)
- Any **non-preemptive** algorithm can experience it under certain workloads.

Interview Answer (20 seconds)

Convoy Effect is a situation where a long CPU-bound process monopolizes the CPU, causing many short processes to wait behind it. This increases average waiting time, reduces responsiveness, and is a major drawback of **FCFS scheduling**.

Context Switch Overhead

What is a Context Switch?

A **context switch** occurs when the CPU stops executing one process/thread and starts executing another. Before switching, the operating system must **save the state** (context) of the current process and **restore the state** of the next process.

What is "Context"?

The context includes information needed to resume a process later, such as:

- Program Counter (PC)
- CPU Registers
- Stack Pointer (SP)
- Process State
- CPU Scheduling Information (stored in the PCB)

Steps in a Context Switch



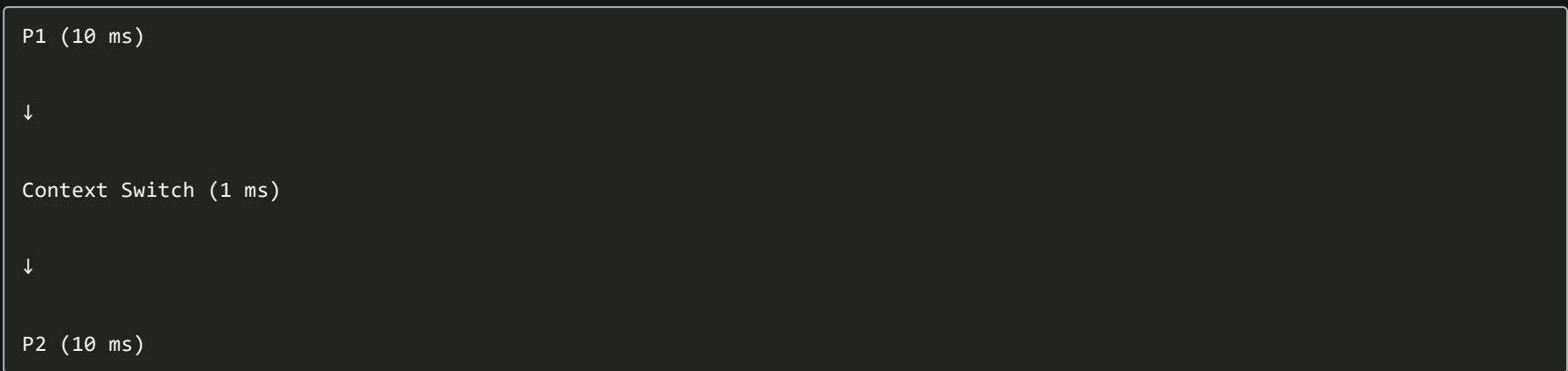
What is Context Switch Overhead?

Context Switch Overhead is the **CPU time spent saving and restoring process state instead of executing user code**. During this time, the CPU is busy with OS work, so **no useful work is done for user processes**.

Why is it an Overhead?

Suppose:

- Running a process takes **10 ms**
- Context switch takes **1 ms**



That **1 ms** is overhead because it doesn't contribute to the actual computation.

Effects of High Context Switching

- Decreases CPU efficiency
- Reduces throughput
- Increases execution time
- Wastes CPU cycles

Which Scheduling Algorithms Have More Overhead?

Algorithm	Context Switch Overhead
FCFS	Low
SJF	Low
Priority (Non-Preemptive)	Low
SRTF	High
Round Robin	High
MLFQ	High

Preemptive algorithms generally have **more context switches**, so they incur **higher overhead**.

Time Quantum and Overhead (Round Robin)

If the **time quantum is too small**:

- Processes are switched very frequently.
- Context-switch overhead becomes high.

If the **time quantum is too large**:

- Fewer context switches.
- Round Robin starts behaving like FCFS, reducing responsiveness.

Interview Answer (30 seconds)

Context switch overhead is the CPU time spent saving the state of the currently running process and restoring the state of the next process during a context switch. Since the CPU performs operating system tasks instead of executing user processes during this time, it is considered overhead. Frequent context switches improve responsiveness but reduce CPU efficiency due to increased overhead.



Throughput & CPU Utilization

These are common interview terms.

Throughput

Number of processes completed per unit time.

Example:

- 100 processes completed in 10 seconds
- Throughput = 10 processes/second

CPU Utilization

Percentage of time the CPU is doing useful work instead of being idle. Higher utilization is generally desirable, but maximizing it alone isn't always the goal if responsiveness suffers.

Starvation

Definition

Starvation (also called **indefinite blocking**) occurs when a process **waits indefinitely** because other processes keep getting the CPU before it. The process is **ready to run**, but it never gets scheduled

Aging

Definition

Aging is a technique used to **prevent starvation** by gradually increasing the priority of a process the longer it waits. Eventually, even low-priority processes become high enough in priority to get the CPU.